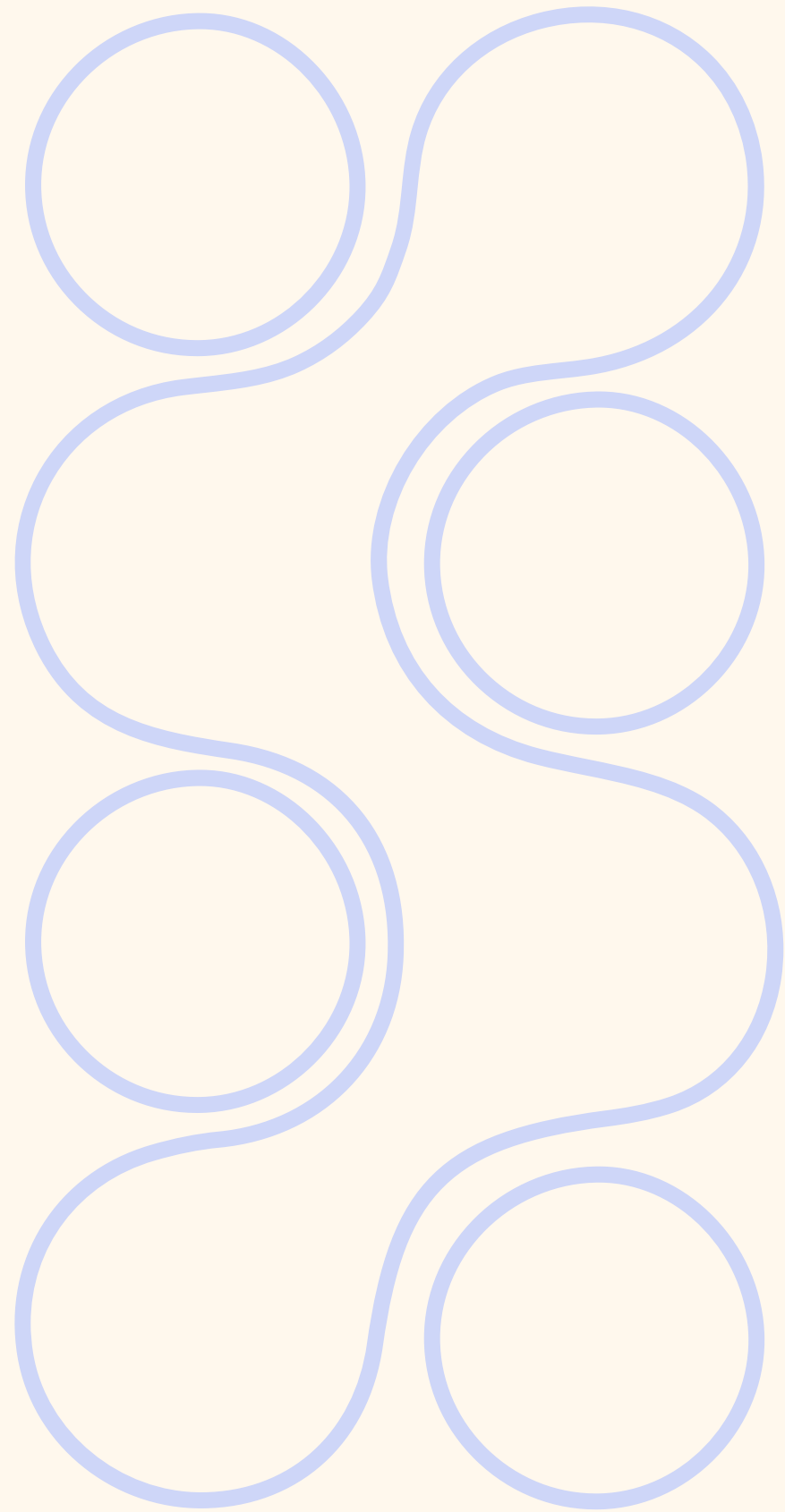




RAPPORT

ATELIER5 : EXAMEN DE FIN DE
FORMATION

Exercice 1:

1. Création de la base et de la collection avec insertion du document

Création de la base **DBSalaries**:

Create Database

Database Name

DBSalaries

```
> use DBSalaries  
< switched to db DBSalaries
```

Création de Collection **salaries**:

Collection Name

salaries

insertion du document :

```
db.salaries.insertOne({  
_id: "s1",nomsal: "Alami",prenomsal: "Sara", fonction: "Technicien",service:  
{codeser: "1",nomser: "informatique"}});
```

```
> db.salaries.insertOne({  
  _id: "s1",  
  nomsal: "Alami",  
  prenomsal: "Sara",  
  fonction: "Technicien",  
  service: {codeser: "1",nomser: "informatique"}});  
< {  
  acknowledged: true,  
  insertedId: 's1'  
}
```

2. Afficher les salariés triés par ordre croissant des noms

db.salaries.find().sort({ nomsal: 1 });

```
> db.salaries.find().sort({ nomsal: 1 });  
< {  
  _id: 's1',  
  nomsal: 'Alami',  
  prenomsal: 'Sara',  
  fonction: 'Technicien',  
  service: {  
    codeser: '1',  
    nomser: 'informatique'
```

3. Afficher le nombre de salariés ayant la fonction "Technicien"

db.salaries.find({ fonction: "Technicien" }).count();

```
> db.salaries.find({ fonction: "Technicien" }).count();  
< 1
```

4. Supprimer le salarié ayant l'_id "s3"

```
db.salaries.deleteOne({ _id: "s3" });
```

```
> db.salaries.deleteOne({ _id: "s3" });  
< {  
  acknowledged: true,  
  deletedCount: 0
```

5. Afficher le nombre de salariés par fonction

```
db.salaries.aggregate([ {  
  $group: {  
    _id: "$fonction"  
  },  
  nombreSalaries: { $sum: 1 } } ])
```

```
> db.salaries.aggregate([  
  {  
    $group: {  
      _id: "$fonction",  
      nombreSalaries: { $sum: 1 }  
    }  
  }  
)  
< {  
  _id: 'Technicien',  
  nombreSalaries: 1  
}
```


Exercice 2 :

1. Création de la base et de la collection avec insertion du document

Création de la base **agricole_lotissement**:

Create Database ✕

Database Name

agricole_lotissement

```
> use agricole_lotissement  
< switched to db agricole_lotissement
```

Création de Collection **terrains**:

Collection Name

terrains

insertion du document :

```
> db.terrains.insertOne ({
  "numT": "T1",
  "type": "agricole",
  "prix": 200000,
  "titre_foncier": "TF123",
  "cin": "P1",
  "adresse": "Rabat",
  "superficie": 300,
  "notaire": {
    "Numn": "N2",
    "nom": "Karimi",
    "prenom": "Omar",
    "adresse": "Rabat",
    "tel": "0612345678"
  }
})
< {
  acknowledged: true,
```

2. Afficher les notaires qui travaillent à "Rabat", triés par nom croissant

```
db.terrains.find(  
  { "notaire.adresse": "Rabat" }, { notaire: 1, _id: 0 }).sort({ "notaire.nom": 1 });
```

```
> db.terrains.find(  
  { "notaire.adresse": "Rabat" },  
  { notaire: 1, _id: 0 }  
) .sort({ "notaire.nom": 1 });  
< {  
  notaire: {  
    Numn: 'N2',  
    nom: 'Karimi',  
    prenom: 'Omar',  
    adresse: 'Rabat',  
    tel: '0612345678'  
  }  
}
```


3. Afficher les notaires qui travaillent à "Rabat", triés par nom croissant

```
db.terrains.aggregate([  
  { $group: { _id: "$type", nombreTerrains: { $sum: 1 } } } ])
```

```
> db.terrains.aggregate([  
  {  
    $group: {  
      _id: "$type",  
      nombreTerrains: { $sum: 1 } } } ])  
< {  
  _id: null,  
  nombreTerrains: 1  
}  
{  
  _id: 'agricole',  
  nombreTerrains: 1  
}
```

4. Afficher le prix maximum des terrains de type agricole

```
db.terrains.aggregate([  
  $match: { "type": "agricole" },  
  {$group: { _id: null,prixMaximum: { $max: "$prix" }}}])
```

```
> db.terrains.aggregate([  
  {$match: { "type": "agricole" }  
  },{$group: { _id: null,prixMaximum: { $max: "$prix" }}}])  
< {  
  _id: null,  
  prixMaximum: 200000  
}
```

5. Augmenter de 1% le prix des terrains agricoles des propriétaires cin = P1 ou cin = P4

```
db.terrains.updateMany({  
  "type": "agricole",  
  "cin": { $in: ["P1", "P4"] },  
  {$mul: { "prix": 1.01 }})
```

```
> db.terrains.updateMany({  
  "type": "agricole",  
  "cin": { $in: ["P1", "P4"] },  
  {$mul: { "prix": 1.01 }})  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

6. Supprimer l'adresse et le téléphone du notaire dont Numn = N2

```
db.terrains.updateMany( { "notaire.Numn": "N2" },  
{ $unset: { "notaire.adresse": "", "notaire.tel": "" } });
```

```
> db.terrains.updateMany(  
  { "notaire.Numn": "N2" },  
  { $unset: { "notaire.adresse": "", "notaire.tel": "" } }  
);  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```